

NuGet與常用套件

李東霖 博士

國立臺灣海洋大學 電機工程學系

2025 Fall semester

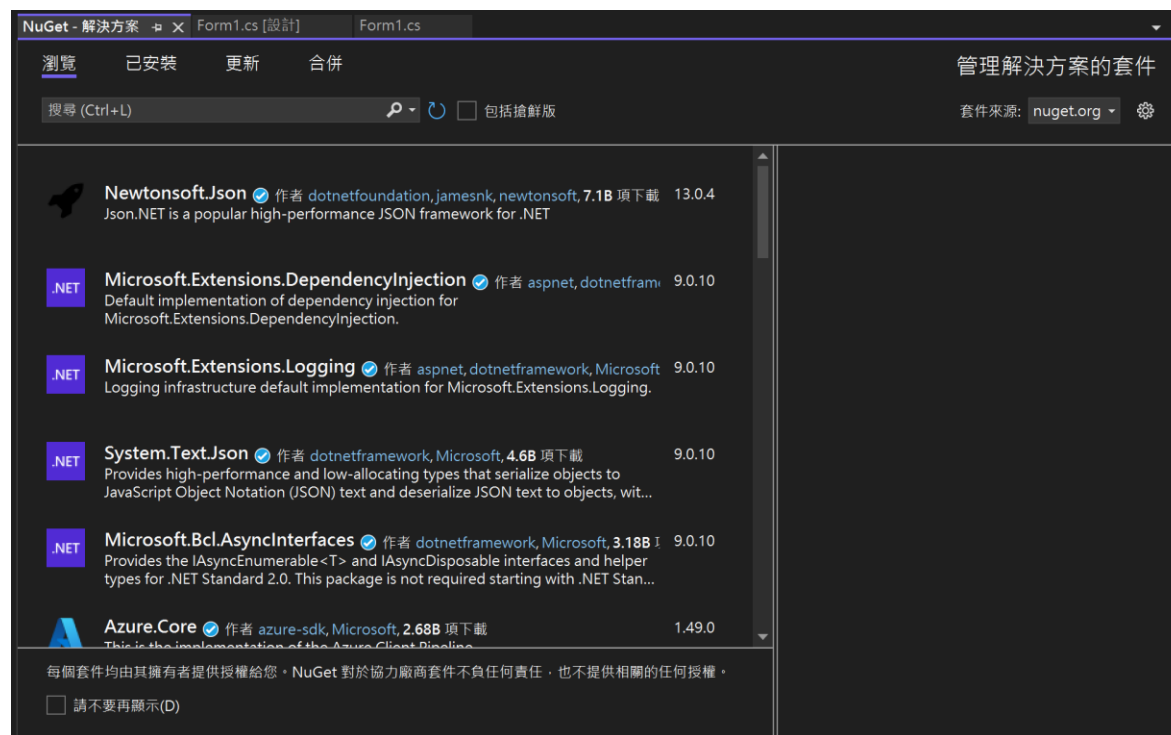
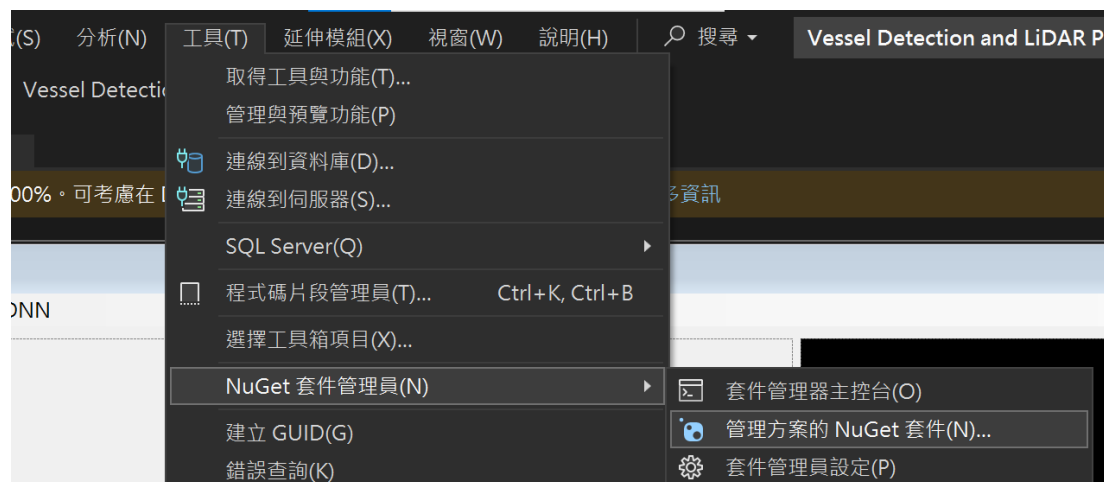
NuGet 簡介

- NuGet 是 .NET 平台的 套件管理系統（ Package Manager ），類似於：
 - Python 的 pip
 - JavaScript 的 npm
 - Java 的 Maven
- 它的主要功能是：
 - 幫助開發者安裝、更新、移除與管理第三方或自訂的 .NET 套件。
 - 自動處理套件之間的依賴關係（ dependencies ）。
 - 支援在 Visual Studio 、 Visual Studio Code 、 或使用 命令列 (dotnet CLI / nuget CLI) 進行操作。
- 套件來源通常是 NuGet 官方套件庫 (nuget.org) ，也可建立公司內部的私有套件伺服器。

NuGet 套件生態地圖



NuGet的安裝方式



目前最多人下載使用的NuGet套件

排名	套件名稱	主要用途	大致下載量
1	Newtonsoft.Json (Json.NET)	JSON 序列化 / 反序列化	約 70 億次以上
2	Microsoft.Extensions.DependencyInjection	.NET 官方依賴注入 DI 模組	約 56 億次
3	Microsoft.Extensions.Logging	.NET 官方日誌系統基礎模組	約 53 · 9 億次
4	System.Text.Json	微軟官方 JSON 處理模組（高效版）	約 45 · 9 億次
5	Microsoft.Bcl.AsyncInterfaces	提供 IAsyncEnumerable / IAsyncDisposable 等介面，支援 .NET 標準	約 31 · 7 億次
6	Azure.Core	微軟 Azure SDK 核心模組	約 26 · 8 億次
7	Serilog	結構化日誌框架	約 21 · 4 億次
8	Microsoft.Win32.SystemEvents	提供 Windows 系統事件 API 封裝	約 20 · 8 億次
9	Microsoft.Identity.Client	微軟身份驗證 MSAL.NET 模組	約 19 · 0 億次
10	Microsoft.EntityFrameworkCore	微軟 ORM：Entity Framework Core	約 17 · 7 億次

Newtonsoft.Json

- Newtonsoft.Json 是 .NET 平台上最受歡迎的 JSON 處理套件。它由 James Newton-King 開發，用於：
 - JSON 資料的序列化（Serialization）：將物件轉成 JSON 字串
 - 反序列化（Deserialization）：將 JSON 字串還原成物件
 - 支援高度客製化的 JSON 處理（格式控制、屬性名稱轉換、自訂轉換器等）
- 主要功能特色：

功能	說明
高速序列化	比 .NET 內建 System.Text.Json 更早出現，長期被使用於企業專案
高度客製化	支援屬性名稱控制、忽略欄位、日期格式化、自訂轉換器
JToken / JObject	可動態解析 JSON 結構，不需事先定義類別
LINQ to JSON	支援用 LINQ 查詢與修改 JSON 資料
廣泛相容	相容於 .NET Framework、.NET Core、.NET 6+、Unity 等環境

JSON的簡介

- JSON（JavaScript Object Notation）是一種用來儲存與交換資料的 輕量級文字格式。
- 它的語法簡單、可讀性高，並且被幾乎所有程式語言支援（C#、Python、Java、JavaScript、Go、Rust...）。
- 常見用途：
 - 伺服器與前端之間的資料交換（API）
 - 設定檔儲存（例如：appsettings.json）
 - 資料序列化（將物件轉成文字傳輸）

JSON 的基本結構

- JSON 由兩種結構組成：

結構	說明	對應於 C# 型別
物件 (Object)	由 {} 包圍，包含多個「鍵值對 (key-value pairs) 」	class 或 Dictionary<string, object>
陣列 (Array)	由 [] 包圍，包含多個元素	List<T> 或 T[]

```
{
  "name": "Alice",
  "age": 25,
  "isStudent": false,
  "skills": ["C#", "Python", "SQL"],
  "address": {
    "city": "Taipei",
    "zipcode": "100"
  }
}
```

"name", "age", "isStudent" 為 屬性名稱 (keys)

對應的值可以是：

- 字串："Alice"
- 整數：25
- 布林值：false
- 陣列：["C#", "Python", "SQL"]
- 物件：{"city": "Taipei", "zipcode": "100"}

JSON 常見資料型別

型別	範例	備註
字串 (string)	"Hello"	需加雙引號
整數 (number)	123	不加引號
布林 (boolean)	true / false	小寫
空值 (null)	null	表示沒有值
陣列 (array)	[1, 2, 3]	
物件 (object)	{"key": "value"}	

JSON 的基本結構

結構	對應於 C# 型別
物件 (Object)	class 或 Dictionary<string, object>
陣列 (Array)	List<T> 或 T[]
字串 (String)	string
數字 (Number)	int, long, float, double...
布林 (Boolean)	bool
空值 (Null)	object

JSON 的應用範例

API 回應 (Response) :

```
{
  "status": "success",
  "data": {
    "user": "Alice",
    "id": 101
  }
}
```

應用程式設定檔 (appsettings.json) :

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information"
    }
  },
  "ConnectionStrings": {
    "DefaultConnection":
      "Server=.;Database=TestDB;Trusted_Connection=True;"
  }
}
```

Newtonsoft.Json範例(1): 序列化與反序列化

```
public class Address
{
    public string City { get; set; }
    public string Street { get; set; }
}
```

```
public class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
    public List<string> Hobbies { get; set; }
    public Address HomeAddress { get; set; }
}
```

```
private void button1_Click(object sender, EventArgs e)
{
    var person = new Person
    {
        Name = "Bob",
        Age = 35,
        Hobbies = new List<string> { "Reading", "Cycling", "Cooking" },
        HomeAddress = new Address
        {
            City = "Taipei",
            Street = "Xinyi Rd. Sec. 5"
        }
    };
};
```

Form1

序列化結果:

```
{
  "Name": "Bob",
  "Age": 35,
  "Hobbies": [
    "Reading",
    "Cycling",
    "Cooking"
  ],
  "HomeAddress": {
    "City": "Taipei",
    "Street": "Xinyi Rd. Sec. 5"
  }
}
```

反序列化結果: Name: BobCity: TaipeiHobbies: Reading, Cycling, Cooking

加入

// 將巢狀物件轉成 JSON 字串

```
string json = JsonConvert.SerializeObject(person, Newtonsoft.Json.Formatting.Indented);
richTextBox1.AppendText("序列化結果:\n" + json);
```

// 將 JSON 還原為物件

```
var person2 = JsonConvert.DeserializeObject<Person>(json);
richTextBox1.AppendText("\n反序列化結果:");
richTextBox1.AppendText($"Name: {person2.Name}");
richTextBox1.AppendText($"City: {person2.HomeAddress.City}");
richTextBox1.AppendText($"Hobbies: " + string.Join(", ", person2.Hobbies));
```

Newtonsoft.Json範例(2): 自訂屬性名稱、忽略屬性、日期格式化

```
public class Employee
{
    // 改輸出名稱為 "full_name"
    [JsonProperty("full_name")]
    public string Name { get; set; }

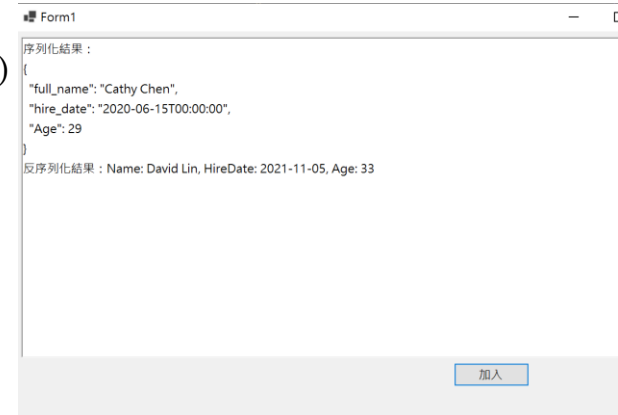
    [JsonProperty("hire_date")]
    // 使用 ISO 日期格式
    [JsonConverter(typeof(IsoDateTimeConverter))]
    public DateTime HireDate { get; set; }

    [JsonIgnore] // 序列化時忽略此屬性
    public string Password { get; set; }

    public int Age { get; set; }
}
```

```
private void button1_Click(object sender, EventArgs e)
{
    var employee = new Employee
    {
        Name = "Cathy Chen",
        Age = 29,
        HireDate = new DateTime(2020, 6, 15),
        Password = "SuperSecret123"
    };
    // 序列化 (格式化輸出)
    string json = JsonConvert.SerializeObject(employee, Newtonsoft.Json.
Formatting.Indented);
    richTextBox1.AppendText("序列化結果：\n" + json);

    // 反序列化回物件
    var jsonInput = @"{
        ""full_name"": ""David Lin"",
        ""hire_date"": ""2021-11-05T00:00:00"",
        ""Age"": 33
    }";
    var emp2 = JsonConvert.DeserializeObject<Employee>(jsonInput);
    richTextBox1.AppendText("\n反序列化結果：");
    richTextBox1.AppendText($"Name: {emp2.Name}, HireDate: {emp2.HireDate:yyyy-MM-
dd}, Age: {emp2.Age}");
}
```



作業練習 - 嘗試將抓到的JSON資料以 ListView表格方式秀出

```
public async Task CrawlerStockByDate(DateTime date)
{
    using (var client = new HttpClient())
    {
        string url = $"https://www.twse.com.tw/exchangeReport/STOCK_DAY?date={date.ToString("yyyyMMdd")}&stockNo=2330";
        string response = await client.GetStringAsync(url);
        var resDatas = JsonConvert.SerializeObject(response);
    }
}
```

```
private void button1_Click(object sender, EventArgs e)
{
    DateTime date = DateTime.Now;
    var task = CrawlerStockByDate(date);
}
```

Microsoft.Extensions.DependencyInjection

- Microsoft.Extensions.DependencyInjection 是 .NET 的 依賴注入 (Dependency Injection, DI) 框架。
- 它是 .NET Core 與 .NET 6/7/8 內建的 IoC 容器 (Inversion of Control Container)，負責自動建立與管理物件的生命週期，讓程式更容易維護、測試與擴充。

IoC與DI

- IoC是用來減低電腦代碼之間的耦合度。其中最常見的方式叫做依賴注入（**Dependency Injection**，簡稱**DI**）
 - 例如:「模組都必須依賴於抽象」是IoC的一種表現，「實現必須依賴抽象，而不是抽象依賴實現」也是IoC的一種表現
- DI是指將一個程式的相依關係改由呼叫它的外部程式來決定的方法。
 - 保留抽象介面，讓組件依賴於抽象介面，當組件要與其它實際的物件發生依賴關係時，藉過抽象介面來注入依賴的實際物件。

使用 DI 的優缺點

- 優點
 1. 低耦合，高內聚
 2. 更清晰的意圖
 3. 降低邏輯變更造成改動程式碼的幅度
 4. 關注點分離，提高可測試性
- 缺點：
 - 系統架構的複雜度增加
 - 我們在寫Code時經常依賴IDE查找引用的功能，對不了解的新手來說，可能會發生找不到實際邏輯到底寫在哪裡的情況，需要多花時間來熟悉框架流程才能上手。

DI的使用

- 常見的注入方式大約可以分為下列三種：
 1. Interface injection
 2. Setter injection
 3. Constructor injection

DI的使用範例(1) - 原始

```
public class UserController : Controller
{
    [HttpGet]
    public IActionResult GetUserList()
    {
        UserRepository userRepository = new UserRepository();
        var userList = userRepository.GetUserList();
        return Json(userList);
    }
}
```

DI的使用範例(2) - 改用注入的方式

```
public class UserController : Controller
{
    private UserRepository _userRepository;

    // Constructor injection
    public UserController(UserRepository userRepository) {
        this._userRepository = userRepository;
    }

    // Setter injection
    public void SetUserRepository(UserRepository userRepository) {
        this._userRepository = userRepository;
    }

    [HttpGet]
    public IActionResult GetUserList()
    {
        var userList = _userRepository.GetUserList();
        return Json(userList);
    }
}
```

Microsoft.Extensions.DependencyInjection的使用範例(1)

```
// 定義介面
public interface IMessageService
{
    void Send(string message);
}

// 實作介面
public class EmailMessageService : IMessageService
{
    public void Send(string message)
    {
        MessageBox.Show($"Sending Email: {message}");
    }
}
```

Microsoft.Extensions.DependencyInjection的使用範例(2)

```
// 另一個類別需要用到 IMessageService
public class NotificationManager
{
    private readonly IMessageService _messageService;

    // 透過建構式注入 (Constructor Injection)
    public NotificationManager(IMessageService messageService)
    {
        _messageService = messageService;
    }

    public void Notify(string msg)
    {
        _messageService.Send(msg);
    }
}
```

Microsoft.Extensions.DependencyInjection的使用範例(3)

```
private void button1_Click(object sender, EventArgs e)
{
    // 1.建立 DI 容器
    var services = new ServiceCollection();

    // 2.註冊服務 (可指定生命週期)
    services.AddTransient<IMessageService, EmailMessageService>();
    services.AddTransient<NotificationManager>();

    // 3.建立 ServiceProvider
    var provider = services.BuildServiceProvider();

    // 4.從容器中取得物件，自動注入相依性
    var manager = provider.GetService<NotificationManager>();
    manager.Notify("Welcome to .NET Dependency Injection!");
}
```